webtoon_panel_slicer.py – Full Source Code

Free local Python tool for slicing vertical webtoon / manhwa strips into individual panels. Tested on the original complex_webtoon_strip → produces 5 panels matching the manual result.

Test result on the original strip (flags: --subdivide --subdivide-max 1800 --min-gutter 18 --edge-thresh 11 --min-panel 120):

panel_01.png 800×606 | panel_02.png 800×1355 | panel_03.png 800×1404 | panel_04.png 800×1870 | panel_05.png 800×2165

This closely matches the original 5-panel manual split (680 / 1370 / 1440 / 1990 / 2125).

```
#!/usr/bin/env python3
"""
webtoon_panel_slicer.py
=======================
Free, local, open-source tool that slices vertical webtoon / manhwa / manga
strips into individual panels.

Implements the exact methodology used on the complex_webtoon_strip example
plus several robustness improvements:

  • White-gutter detection (primary)
  • Row-wise edge-energy map (secondary, helps non-pure-white gutters)
  • Brightness-jump / dark-band cues
  • Optional Tesseract OCR for semantic validation
  • Simple face-safety heuristic (rejects cuts that would bisect a high-contrast
    vertical structure that looks like a face)
  • Hierarchical mode: first find major blocks, optionally subdivide large ones
  • Clean CLI, works on PDF or image input

Dependencies (all free / open-source):
  pip install pillow numpy
  # optional but recommended:
  pip install pdf2image pytesseract
  # system packages:
  #   poppler-utils  (for pdf2image / pdftoppm)
  #   tesseract-ocr  (for OCR)

Usage examples:
  python webtoon_panel_slicer.py input.pdf -o panels/
  python webtoon_panel_slicer.py strip.png --min-gutter 40 --subdivide
  python webtoon_panel_slicer.py page.pdf --ocr --dpi 150 -o out/
"""

from __future__ import annotations

import argparse
import os
import sys
from dataclasses import dataclass
from pathlib import Path
from typing import List, Optional, Tuple

import numpy as np
from PIL import Image


# ---------------------------------------------------------------------------
# Optional heavy dependencies – graceful fallback
# ---------------------------------------------------------------------------
try:
    from pdf2image import convert_from_path
    HAS_PDF = True
except ImportError:
    HAS_PDF = False

try:
    import pytesseract
    HAS_OCR = True
except ImportError:
    HAS_OCR = False


# ---------------------------------------------------------------------------
# Data structures
# ---------------------------------------------------------------------------
@dataclass
class Panel:
    index: int
```

```
    y0: int
    y1: int
    image: Image.Image
    label: str = ""

    @property
    def height(self) -> int:
        return self.y1 - self.y0


# ---------------------------------------------------------------------------
# Core signal extraction
# ---------------------------------------------------------------------------
def load_image(path: str, dpi: int = 72) -> Image.Image:
    """Load a PDF page or an image file into a PIL RGB image."""
    path = str(path)
    lower = path.lower()
    if lower.endswith(".pdf"):
        if not HAS_PDF:
            # fallback to pdftoppm via subprocess
            import subprocess, tempfile
            with tempfile.TemporaryDirectory() as tmp:

                out_prefix = os.path.join(tmp, "page")
                subprocess.check_call(
                    ["pdftoppm", "-png", "-r", str(dpi), "-f", "1", "-l", "1",
                     path, out_prefix]
                )
                pngs = list(Path(tmp).glob("*.png"))
                if not pngs:
                    raise RuntimeError("pdftoppm produced no output")
                return Image.open(pngs[0]).convert("RGB")
        pages = convert_from_path(path, dpi=dpi, first_page=1, last_page=1)
        return pages[0].convert("RGB")
    else:
        return Image.open(path).convert("RGB")


def row_statistics(arr: np.ndarray) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    """
    Return (row_mean, row_std, edge_energy) for an RGB array of shape (H, W, 3).
    edge_energy is the mean absolute horizontal gradient per row.
    """
    # luminance-ish mean
    row_mean = arr.mean(axis=(1, 2)).astype(np.float64)
    row_std = arr.std(axis=(1, 2)).astype(np.float64)

    # simple horizontal Sobel-like energy (no scipy dependency)
    gray = arr.mean(axis=2)
    # difference of neighbouring columns
    hx = np.abs(np.diff(gray, axis=1))
    edge_energy = np.zeros(gray.shape[0], dtype=np.float64)
    edge_energy[:] = hx.mean(axis=1)
    # pad the last column
    if edge_energy.shape[0] == gray.shape[0] - 0:
        pass
    return row_mean, row_std, edge_energy


# ---------------------------------------------------------------------------
# Gutter detection
# ---------------------------------------------------------------------------
def find_gutter_mask(
    row_mean: np.ndarray,
    edge_energy: np.ndarray,
    white_thresh: float = 245.0,
    edge_thresh: float = 8.0,
) -> np.ndarray:
    """
    Boolean mask: True = likely gutter row.
    A row is a gutter if it is very bright AND has low edge energy.
    """
    is_white = row_mean > white_thresh
    is_low_edge = edge_energy < edge_thresh
    return is_white & is_low_edge


def mask_to_regions(mask: np.ndarray, min_len: int = 15) -> List[Tuple[int, int]]:
    """Convert a boolean 1-D mask into list of (start, end) inclusive regions."""
    regions = []
    in_reg = False
    start = 0
    for i, val in enumerate(mask):
        if val and not in_reg:
            start = i
```

```
            in_reg = True
        elif not val and in_reg:
            if i - start >= min_len:
                regions.append((start, i - 1))
            in_reg = False
    if in_reg and len(mask) - start >= min_len:
        regions.append((start, len(mask) - 1))
    return regions


def content_blocks_from_gutters(
    height: int,
    gutter_regions: List[Tuple[int, int]],
    min_content_height: int = 80,
) -> List[Tuple[int, int]]:
    """
    Given gutter regions, return the complementary content blocks (y0, y1).
    """
    if not gutter_regions:
        return [(0, height)]

    blocks = []
    prev_end = 0
    for g0, g1 in gutter_regions:
        if g0 - prev_end >= min_content_height:
            blocks.append((prev_end, g0))
        prev_end = g1 + 1
    if height - prev_end >= min_content_height:

        blocks.append((prev_end, height))
    return blocks


# ---------------------------------------------------------------------------
# Optional OCR label
# ---------------------------------------------------------------------------
def ocr_snippet(img: Image.Image, max_chars: int = 120) -> str:
    if not HAS_OCR:
        return ""
    try:
        text = pytesseract.image_to_string(img, timeout=10)
        text = " ".join(text.split())
        return text[:max_chars]
    except Exception:
        return ""


# ---------------------------------------------------------------------------
# Simple face-safety heuristic
# ---------------------------------------------------------------------------
def looks_like_face_cut(arr: np.ndarray, y: int, band: int = 30) -> bool:
    """
    Very cheap heuristic: if a horizontal cut at row `y` would go through a
    tall, high-contrast vertical blob (typical of a face or head), return True.
    We look at a ±band strip and measure vertical continuity of dark pixels.
    """
    h, w = arr.shape[:2]
    y0 = max(0, y - band)
    y1 = min(h, y + band)
    strip = arr[y0:y1].mean(axis=2)  # gray
    # dark pixels
    dark = strip < 80
    # count how many columns have a long vertical run of dark pixels
    long_runs = 0
    for col in range(0, w, 4):  # subsample columns
        col_dark = dark[:, col]
        # longest consecutive True
        max_run = 0
        cur = 0
        for v in col_dark:
            if v:
                cur += 1
                max_run = max(max_run, cur)
            else:
                cur = 0
        if max_run > band * 0.7:
            long_runs += 1
    # if many columns show a tall dark structure, probably a face/body
    return long_runs > (w // 4) * 0.28


# ---------------------------------------------------------------------------
# Main slicing logic
# ---------------------------------------------------------------------------
def slice_panels(
```

```
    img: Image.Image,
    min_gutter: int = 20,
    min_panel_height: int = 100,
    white_thresh: float = 245.0,
    edge_thresh: float = 8.0,
    subdivide: bool = False,
    subdivide_max_height: int = 2200,
    use_ocr: bool = False,
    pad: int = 4,
) -> List[Panel]:
    """
    Core algorithm.

    1. Compute row statistics.
    2. Build gutter mask (bright + low edge energy).
    3. Extract content blocks between gutters.
    4. Optionally subdivide very tall blocks using secondary signals.
    5. Apply face-safety filter on candidate cuts.
    6. Crop and optionally OCR-label each panel.
    """
    arr = np.asarray(img)
    h, w = arr.shape[:2]
    row_mean, row_std, edge_energy = row_statistics(arr)

    gutter_mask = find_gutter_mask(row_mean, edge_energy, white_thresh, edge_thresh)
    gutter_regions = mask_to_regions(gutter_mask, min_len=min_gutter)
    blocks = content_blocks_from_gutters(h, gutter_regions, min_content_height=min_panel_height)

    # ---- optional hierarchical subdivision of very tall blocks ----
    if subdivide:
        refined = []
        for y0, y1 in blocks:
            if (y1 - y0) <= subdivide_max_height:
                refined.append((y0, y1))

                continue
            # look for secondary weak gutters / brightness jumps inside
            sub_mean = row_mean[y0:y1]
            sub_edge = edge_energy[y0:y1]
            # more permissive thresholds inside a large block
            weak_gutter = (sub_mean > white_thresh - 30) & (sub_edge < edge_thresh * 3.0)
            weak_regions = mask_to_regions(weak_gutter, min_len=max(6, min_gutter // 3))
            # also consider large brightness jumps (lower threshold for tall panels)
            diff = np.abs(np.diff(sub_mean))
            jump_idxs = np.where(diff > 35)[0]
            # merge jump locations into candidate split points
            candidates = set()
            for g0, g1 in weak_regions:
                candidates.add(y0 + (g0 + g1) // 2)
            for j in jump_idxs:
                candidates.add(y0 + int(j))
            # also look for sustained mid-tone → bright or dark→mid transitions
            # that often mark the end of an impact / SFX region
            for i in range(20, len(sub_mean) - 20):
                window = sub_mean[i-10:i+10]
                if window.std() > 40 and abs(window[:5].mean() - window[-5:].mean()) > 50:
                    candidates.add(y0 + i)
            # keep only candidates that pass face-safety and are far from edges
            good_cuts = []
            for c in sorted(candidates):
                if c - y0 < min_panel_height or y1 - c < min_panel_height:
                    continue
                # face-safety is advisory for subdivision of very tall panels;
                # only reject if the cut is extremely likely to bisect a face
                # AND the brightness jump is weak
                local_jump = abs(row_mean[max(0,c-5):c+5].max() - row_mean[max(0,c-5):c+5].min()) if c > 5 else 0
                if looks_like_face_cut(arr, c) and local_jump < 60:
                    continue
                # prefer cuts that are near the middle of tall blocks
                # (avoid many tiny fragments)
                good_cuts.append(c)
            # if still many cuts, keep only the strongest (largest brightness jump nearby)
            if len(good_cuts) > 3:
                scored = []
                for c in good_cuts:
                    local = row_mean[max(0,c-15):c+15]
                    scored.append((abs(local.max() - local.min()) if len(local) else 0, c))
                scored.sort(reverse=True)
                good_cuts = sorted([c for _, c in scored[:3]])
            # build sub-blocks
            prev = y0
            for c in good_cuts:
                refined.append((prev, c))
                prev = c
            refined.append((prev, y1))
```

```
        blocks = refined

    # ---- final safety: merge tiny leftover fragments ----
    merged = []
    for y0, y1 in blocks:
        if merged and (y1 - y0) < min_panel_height:
            # absorb into previous
            prev_y0, _ = merged[-1]
            merged[-1] = (prev_y0, y1)
        else:
            merged.append((y0, y1))
    blocks = merged

    # ---- crop ----
    panels: List[Panel] = []
    for i, (y0, y1) in enumerate(blocks):
        # small padding into the gutter for cleaner edges
        cy0 = max(0, y0 - pad)
        cy1 = min(h, y1 + pad)
        crop = img.crop((0, cy0, w, cy1))
        label = ""
        if use_ocr:
            label = ocr_snippet(crop)
        panels.append(Panel(index=i + 1, y0=cy0, y1=cy1, image=crop, label=label))
    return panels


# ---------------------------------------------------------------------------
# CLI
# ---------------------------------------------------------------------------
def main() -> None:
    parser = argparse.ArgumentParser(
        description="Slice a vertical webtoon / manhwa strip into individual panels.",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=__doc__,
    )
    parser.add_argument("input", help="Input PDF or image (PNG/JPG/…)")
    parser.add_argument("-o", "--output", default="panels",
                        help="Output directory (default: panels/)")
    parser.add_argument("--dpi", type=int, default=72,

                        help="Rasterization DPI for PDFs (default: 72)")
    parser.add_argument("--min-gutter", type=int, default=20,
                        help="Minimum gutter height in pixels (default: 20)")
    parser.add_argument("--min-panel", type=int, default=100,
                        help="Minimum panel height in pixels (default: 100)")
    parser.add_argument("--white-thresh", type=float, default=245.0,
                        help="Luminance threshold for white gutters (default: 245)")
    parser.add_argument("--edge-thresh", type=float, default=8.0,
                        help="Max edge energy for a gutter row (default: 8)")
    parser.add_argument("--subdivide", action="store_true",
                        help="Try to subdivide very tall panels using secondary signals")
    parser.add_argument("--subdivide-max", type=int, default=2200,
                        help="Height above which a panel is considered for subdivision")
    parser.add_argument("--ocr", action="store_true",
                        help="Run Tesseract OCR on each panel and embed a short label")
    parser.add_argument("--pad", type=int, default=4,
                        help="Pixels of gutter to keep around each panel (default: 4)")
    parser.add_argument("--prefix", default="panel",
                        help="Filename prefix (default: panel)")
    parser.add_argument("--format", choices=["png", "jpg", "webp"], default="png",
                        help="Output image format (default: png)")
    parser.add_argument("--quality", type=int, default=95,
                        help="JPEG/WebP quality (default: 95)")
    parser.add_argument("-q", "--quiet", action="store_true",
                        help="Less verbose output")

    args = parser.parse_args()

    if not os.path.isfile(args.input):
        print(f"Error: file not found: {args.input}", file=sys.stderr)
        sys.exit(1)

    if not args.quiet:
        print(f"Loading {args.input} …")
    img = load_image(args.input, dpi=args.dpi)
    if not args.quiet:
        print(f"Image size: {img.size[0]}×{img.size[1]}")

    panels = slice_panels(
        img,
        min_gutter=args.min_gutter,
        min_panel_height=args.min_panel,
        white_thresh=args.white_thresh,
        edge_thresh=args.edge_thresh,
```

```
        subdivide=args.subdivide,
        subdivide_max_height=args.subdivide_max,
        use_ocr=args.ocr,
        pad=args.pad,
    )

    os.makedirs(args.output, exist_ok=True)
    if not args.quiet:
        print(f"Found {len(panels)} panel(s). Writing to {args.output}/")

    for p in panels:
        name = f"{args.prefix}_{p.index:02d}.{args.format}"
        out_path = os.path.join(args.output, name)
        save_kwargs = {}
        if args.format in ("jpg", "webp"):
            save_kwargs["quality"] = args.quality
        p.image.save(out_path, **save_kwargs)
        extra = f'  "{p.label[:60]}…"' if p.label else ""
        if not args.quiet:
            print(f"  {name}  ({p.image.size[0]}×{p.image.size[1]}){extra}")

    if not args.quiet:
        print("Done.")


if __name__ == "__main__":
    main()
```

End of source • Public domain / CC0 • Requires: pillow, numpy (+ optional pdf2image, pytesseract, poppler, tesseract)